Importing the needed libraries

↳ 4 cells hidden

Reading the dataset and exploring it

↳ 6 cells hidden

Creating the methods needed for text mining in order to make the code more effecient

removing the stop words method:

```
stop_words = set(stopwords.words('english'))
def stopword_remove(x):
  return [W for W in x if not W.lower() in stop_words]
```

removing punctuation method:

```
punctuation = set(string.punctuation)
def punctuation_remove(x):
  return [W for W in x if not W in punctuation]
```

Converting the text to lower case method:

```
def convert_lower(x):
  return [W.lower() for W in x]
```

stemming method:

```
st = PorterStemmer()
def stem(x):
  return [st.stem(W) for W in x]
```

combining all of the methods and adding the tokenizatin and joining steps

```
def TM_prep (query):
```

```
    query = word_tokenize(query)

    query = stopword_remove(query)

    query = punctuation_remove(query)

    query = convert_lower(query)

    query = stem(query)

    query = ' '.join(query)
    return query
```

Calculating TF-IDF method

```
vect = TfidfVectorizer()

def TF_IDF_text (text):

  TF_IDF_Matrix = vect.fit_transform(text)
  TF_IDF_Array = TF_IDF_Matrix.toarray()

  for i, bio in enumerate(text):

    TF_IDF = TF_IDF_Array[i]

    dict = {vect.get_feature_names_out()[j]: TF_IDF[j] for j in range

    return dict, TF_IDF_Array, TF_IDF_Matrix
```

```
def TF_IDF_query (query):

  TF_IDF_Matrix = vect.transform(query)
  TF_IDF_Array = TF_IDF_Matrix.toarray()

  for i, bio in enumerate(query):

    TF_IDF = TF_IDF_Array[i]

    dict = {vect.get_feature_names_out()[j]: TF_IDF[j] for j in range

    return dict, TF_IDF_Array, TF_IDF_Matrix
```

printing the TF-IDF values method:

```
def display_TFIDF (text,dict):
  for i, bio in enumerate(text):
      print(f"Calculated TF-IDF for doc {i+1}:")
      for word, tfidf in dict.items():
           print(f"{word}: {tfidf}")
      print()
```

calculating smilirty mesuers method:

```
def similarity_mesuers(query_matrix,Bio_matrix):

  similarity_results = []

  for idx, doc_vec in enumerate(Bio_matrix):
      manhattan_dist = cityblock(query_matrix.toarray()[0], doc_vec.t
      euclidean_dist = euclidean(query_matrix.toarray()[0], doc_vec.t
      if np.linalg.norm(query_matrix.toarray()[0]) == 0 or np.linalg.
        cosine_sim = 0
      else:
        cosine_sim = 1 - cosine(query_matrix.toarray()[0], doc_vec.to

      similarity_results.append({
          'Document': f'bio{idx + 1}',
          'Manhattan Distance': manhattan_dist,
          'Euclidean Distance': euclidean_dist,
          'Cosine Similarity': cosine_sim
      })

  similarity_df = pd.DataFrame(similarity_results)
  return similarity_df
```

Applying Text mining using the TF-IDF teqhnique

↳ 6 cells hidden

Query one

↳ 11 cells hidden

Query two

↳ 11 cells hidden

Query three

↳ 11 cells hidden

Query four

↳ 11 cells hidden

Query five

```
q5 =  "Jordan university AI graduate student."
```

```
q5 = TM_prep(q5)
print(q5)
```

```
jordan univers ai graduat student
```

calculating the TF-IDF

```
query5_dict, query5_TFID_Array, query5_TFID_Matrix = TF_IDF_query([q5
```

```
display_TFIDF(q4,query5_dict)
```

```
methodolog: 0.0
microsoft: 0.0
mindset: 0.0
mine: 0.0
mitig: 0.0
mobil: 0.0
model: 0.0
mohammad: 0.0
mona: 0.0
motiv: 0.0
mousa: 0.0
multipl: 0.0
muna: 0.0
nabil: 0.0
nada: 0.0
nader: 0.0
nadia: 0.0
nadin: 0.0
naji: 0.0
name: 0.0
naseem: 0.0
natur: 0.0
need: 0.0
network: 0.0
new: 0.0
next: 0.0
ngo: 0.0
nisreen: 0.0
noor: 0.0
```

similarity mesuers

```
q5_similarty = similarity_mesuers(query5_TFID_Matrix,Bio_TFIDF_Matrix
q5_similarty
```

	Document	Manhattan Distance	Euclidean Distance	Cosine Similarity
0	bio1	6.631361	1.343408	0.097627
1	bio2	6.851536	1.403397	0.015239
2	bio3	6.455077	1.394279	0.027993
3	bio4	6.537605	1.409448	0.006728
4	bio5	6.552606	1.409142	0.007159
...	...	...	...	...
252	bio253	4.450170	1.357105	0.079134
253	bio254	4.485326	1.403200	0.015515
254	bio255	4.561670	1.315167	0.135168
255	bio256	4.417770	1.405105	0.012840
256	bio257	4.389605	1.358418	0.077351

257 rows × 4 columns

displaying bios that are relavent to the query using cosine similarity

```
q5_cosine_df = q5_similarty.drop(['Manhattan Distance','Euclidean Dis
q5_cosine_df = q5_cosine_df[q5_cosine_df["Cosine Similarity"] > 0.0]
```

```
q5_cosine_df["Document"] = q5_cosine_df["Document"].apply(lambda i:i.
q5_cosine_df["Document"] = q5_cosine_df["Document"].apply(lambda i:in
q5_cosine_df = q5_cosine_df.sort_values("Cosine Similarity", ascendin
q5_cosine_df
```

	Document	Cosine Similarity
15	16	0.246439
73	74	0.239687
87	88	0.227611
10	11	0.205576
46	47	0.205521
...	...	...
50	51	0.007373
39	40	0.007288
4	5	0.007159
29	30	0.006858
3	4	0.006728

257 rows × 2 columns

```
if  not q5_cosine_df.empty:
  for i in q5_cosine_df["Document"]:
      print(df.loc[i-1,"Bio"])
      print()
else:
    print("There are no bios that matches your describtion")
```

```
Yara, a senior at Princess Sumaya University for Technology, brings c

Rami is a third-year Data Science student at the University of Jordan

Lina is a first-year Computer Science student at Princess Sumaya Univ

Hassan's early exposure to Data Science sparked a deep interest in ma

Muna, a student at Jordan University of Science & Technology, is alre

Rasha is a second-year Data Science student at Al Hussein Technical U

Abeer is a Data Science student at Jordan University of Science & Tec

Nour is a third-year Data Science student at Jordan University of Sci

Samar, a Data Science and AI student, is fascinated by the power of a

Kareem is a senior Data Science student at Petra University with a fo

Yousef, a senior at Jordan University of Science & Technology, combin

Salem is a second-year Data Science student at Al Hussein Technical U

Dalia is a second-year Data Science student at Al Hussein Technical U

Nour, a second-year student at Petra University, is fascinated by the
```

```
Jad is a second-year Data Science student at Princess Sumaya Universi
Nabil, a senior Data Science student at Jordan University of Science
Fadi, a second-year student at Al Hussein Technical University, is pa
Omar is a senior Data Science student at Al Hussein Technical Univers
Hassan, a second-year Data Science student at the University of Jorda
Yasmin, a third-year student at the University of Jordan, is passiona
Rami, a senior at Petra University, has a strong foundation in data s
Noor is a second-year Computer Science student at the University of J
Amjad is a Data Science student at the University of Jordan. He has c
Laila has always been fascinated by the patterns data reveals. Now a
Ziad, a second-year Data Science and AI student at Jordan University
Mahmoud, a second-year Data Science student at Jordan University of S
Mahmoud is a second-year Data Science student at Jordan University of
Hala is a third-year Data Science student at Princess Sumaya Universi
Karim's enthusiasm for AI and machine learning drives his academic pu
```

text mining application for a real-world issue

↳ 24 cells hidden

Graph mining

↳ 5 cells hidden

graph mining real life scenario

↳ 11 cells hidden

preparing the dataset for clustering

↳ 8 cells hidden

Feature extraction

↳ 2 cells hidden

PCA

↳ 10 cells hidden

LDA

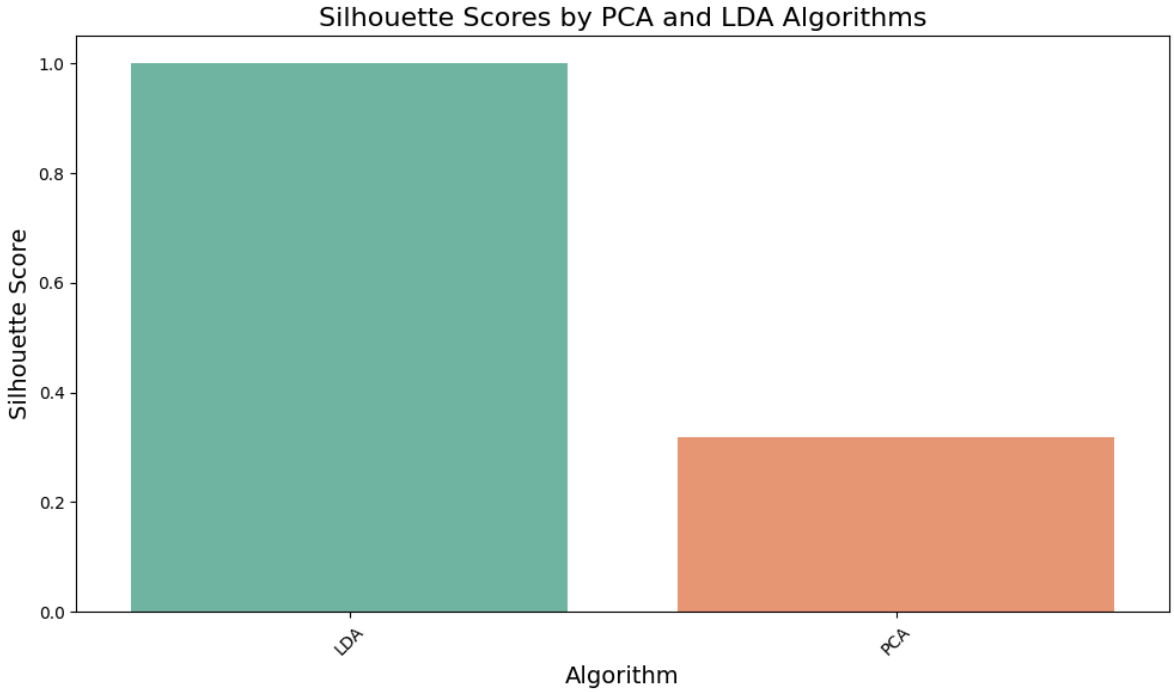
↳ 10 cells hidden

Visualizing the results of both LDA and PCA to compare

Bar plot for Silhouette score

```
plt.figure(figsize=(10, 6))
sns.barplot(x=["LDA","PCA"], y=[silhouette_scores_FE['LDA'][0],silhou
plt.title("Silhouette Scores by PCA and LDA Algorithms", fontsize=16)
plt.ylabel("Silhouette Score", fontsize=14)
plt.xlabel("Algorithm", fontsize=14)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

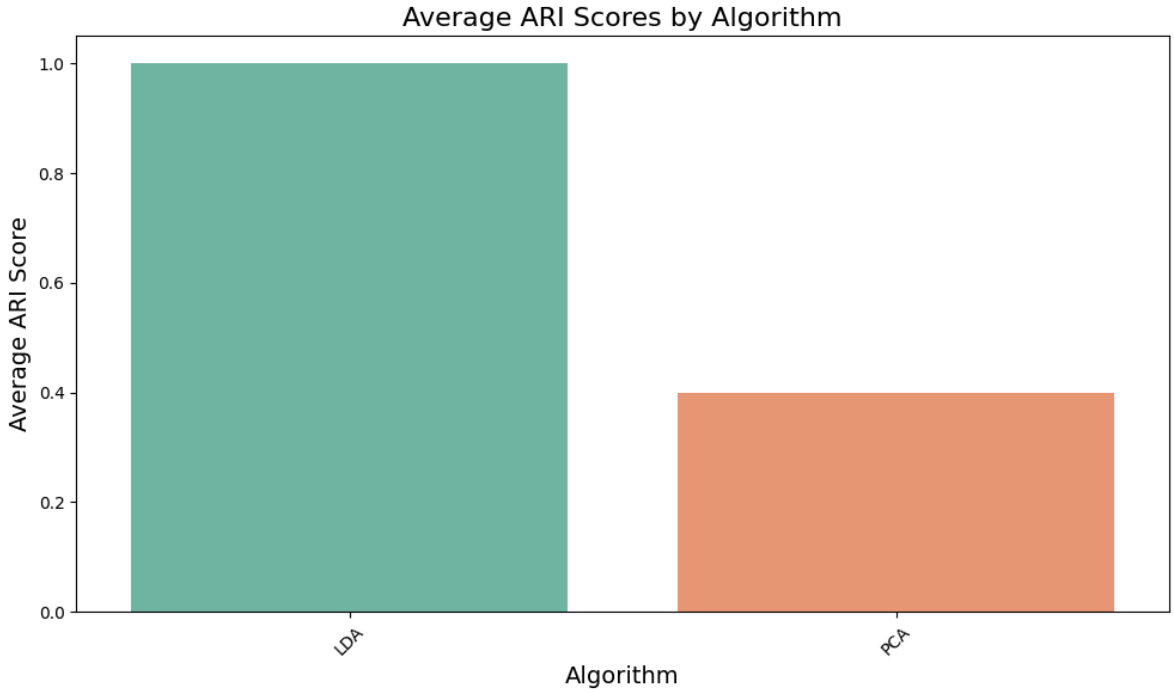
```
<ipython-input-94-da8577d373a2>:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be re

  sns.barplot(x=["LDA","PCA"], y=[silhouette_scores_FE['LDA'][0],silho
```



Bar plot for ARI score

```
plt.figure(figsize=(10, 6))
sns.barplot(x=["LDA","PCA"], y=[ari_scores_FE['LDA'][0],ari_scores_FE
plt.title("Average ARI Scores by Algorithm", fontsize=16)
plt.ylabel("Average ARI Score", fontsize=14)
plt.xlabel("Algorithm", fontsize=14)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
<ipython-input-95-3c9c59f354fd>:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be re

  sns.barplot(x=["LDA","PCA"], y=[ari_scores_FE['LDA'][0],ari_scores_F
```



We can notice here that LDA feature extraction algorithm showed better performance as its scores were higher in all evaluation scores and it scored lower in DB index which shows that it is better as well

clustring Algorithms

creating lists for the scores

```
silhouette_scores = {'kmeans': [],'DBSCAN': [], 'HDBSCAN': [], 'OPTIC
                     'Agglomerative': []}
ari_scores = {'kmeans': [],'DBSCAN': [], 'HDBSCAN': [], 'OPTICS': [],
              'Agglomerative': []}
```

```
CH_scores = {'kmeans': [],'DBSCAN': [], 'HDBSCAN': [], 'OPTICS': [],
             'Agglomerative': []}
```

applying each algorithm 5 times

```
for i in range(5):
    print(f"iteration {i + 1}:")


    kmeans = KMeans()

    kmeans_clusters = kmeans.fit_predict(X_scaled)

    kmeans_silhouette = silhouette_score(X_scaled, kmeans_clusters)

    kmeans_ari = adjusted_rand_score(Y, kmeans_clusters)

    CH_kmeans = calinski_harabasz_score(X_scaled, kmeans_clusters)

    silhouette_scores['kmeans'].append(kmeans_silhouette)

    ari_scores['kmeans'].append(kmeans_ari)

    CH_scores['kmeans'].append(CH_kmeans)


    print(f"kmeans: Silhouette Score: {kmeans_silhouette:.4f}, ARI Sc


    dbscan = DBSCAN()

    dbscan_clusters = dbscan.fit_predict(X_scaled)

    dbscan_silhouette = silhouette_score(X_scaled, dbscan_clusters)

    dbscan_ari = adjusted_rand_score(Y, dbscan_clusters)

    CH_dbscan = calinski_harabasz_score(X_scaled, dbscan_clusters)

    silhouette_scores['DBSCAN'].append(dbscan_silhouette)

    ari_scores['DBSCAN'].append(dbscan_ari)

    CH_scores['DBSCAN'].append(CH_dbscan)

    print(f"DBSCAN: Silhouette Score: {dbscan_silhouette:.4f}, ARI Sc


    hdbscan = HDBSCAN()

    hdbscan_clusters = hdbscan.fit_predict(X_scaled)

    hdbscan_silhouette = silhouette_score(X_scaled, hdbscan_clusters)
```

```
hdbscan_ari = adjusted_rand_score(Y, hdbscan_clusters)
CH_hdbscan = calinski_harabasz_score(X_scaled, hdbscan_clusters)
silhouette_scores['HDBSCAN'].append(hdbscan_silhouette)
ari_scores['HDBSCAN'].append(hdbscan_ari)
CH_scores['HDBSCAN'].append(CH_hdbscan)
print(f"HDBSCAN: Silhouette Score: {hdbscan_silhouette:.4f}, ARI S

optics = OPTICS()
optics_clusters = optics.fit_predict(X_scaled)
optics_silhouette = silhouette_score(X_scaled, optics_clusters)
optics_ari = adjusted_rand_score(Y, optics_clusters)
CH_optics = calinski_harabasz_score(X_scaled, optics_clusters)
silhouette_scores['OPTICS'].append(optics_silhouette)
ari_scores['OPTICS'].append(optics_ari)
CH_scores['OPTICS'].append(CH_optics)

print(f"OPTICS: Silhouette Score: {optics_silhouette:.4f}, ARI Sc

bisect_kmeans = BisectingKMeans(n_clusters=3)
bisect_kmeans_clusters = bisect_kmeans.fit_predict(X_scaled)
bisect_kmeans_silhouette = silhouette_score(X_scaled, bisect_kmea
bisect_kmeans_ari = adjusted_rand_score(Y, bisect_kmeans_clusters
CH_bisect_kmeans = calinski_harabasz_score(X_scaled, bisect_kmean
silhouette_scores['BisectKMeans'].append(bisect_kmeans_silhouette
ari_scores['BisectKMeans'].append(bisect_kmeans_ari)
CH_scores['BisectKMeans'].append(CH_bisect_kmeans)
print(f"BisectKMeans: Silhouette Score: {bisect_kmeans_silhouette

agglomerative = AgglomerativeClustering(n_clusters=4)
```

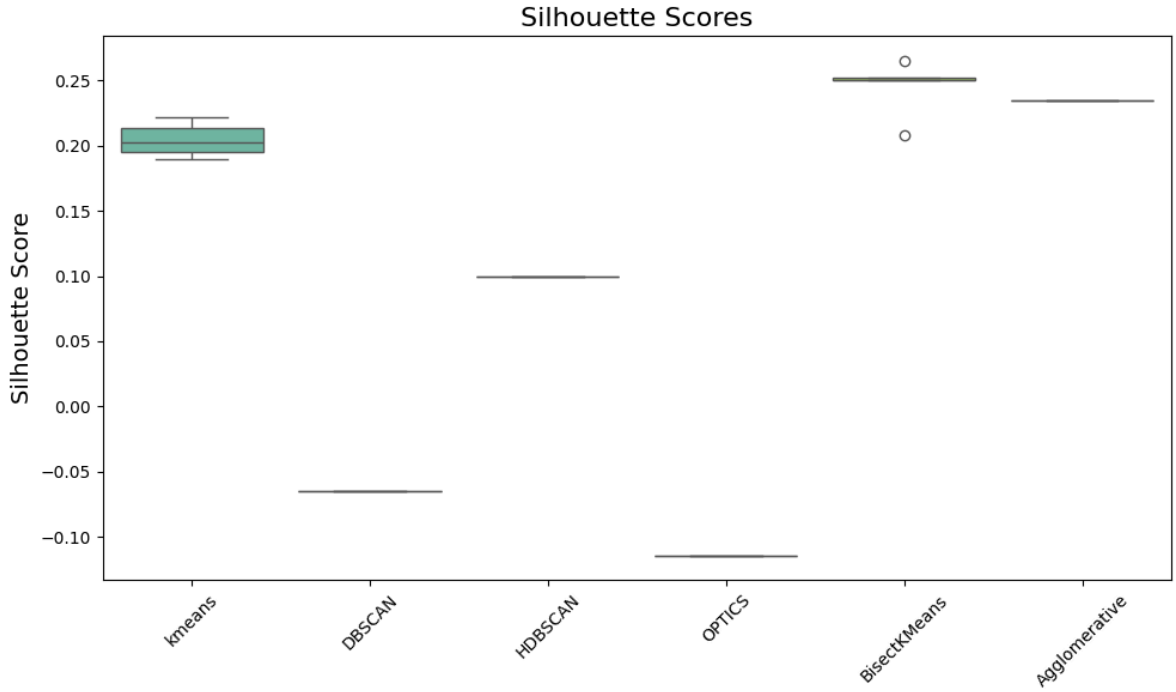
```
    agglomerative_clusters = agglomerative.fit_predict(X_scaled)

    agglomerative_silhouette = silhouette_score(X_scaled, agglomerati

    agglomerative_ari = adjusted_rand_score(Y, agglomerative_clusters

    CH_agglomerative = calinski_harabasz_score(X_scaled, agglomerativ

    silhouette_scores['Agglomerative'].append(agglomerative_silhouett

    ari_scores['Agglomerative'].append(agglomerative_ari)

    CH_scores['Agglomerative'].append(CH_agglomerative)

    print(f"AgglomerativeClustering: Silhouette Score: {agglomerative_
```

```
iteration 1:
kmeans: Silhouette Score: 0.1991, ARI Score: 0.2632,CH Score: 53.4248
DBSCAN: Silhouette Score: -0.0647, ARI Score: 0.0000,CH Score: 4.2651
HDBSCAN: Silhouette Score: 0.0993, ARI Score: 0.4836,CH Score: 31.8941
OPTICS: Silhouette Score: -0.1142, ARI Score: 0.0276,CH Score: 8.5230
BisectKMeans: Silhouette Score: 0.2523, ARI Score: 0.4319,CH Score: 96
AgglomerativeClustering: Silhouette Score: 0.2350, ARI Score: 0.4847,
iteration 2:
kmeans: Silhouette Score: 0.2224, ARI Score: 0.4229,CH Score: 60.2906
DBSCAN: Silhouette Score: -0.0647, ARI Score: 0.0000,CH Score: 4.2651
HDBSCAN: Silhouette Score: 0.0993, ARI Score: 0.4836,CH Score: 31.8941
OPTICS: Silhouette Score: -0.1142, ARI Score: 0.0276,CH Score: 8.5230
BisectKMeans: Silhouette Score: 0.2502, ARI Score: 0.4427,CH Score: 95
AgglomerativeClustering: Silhouette Score: 0.2350, ARI Score: 0.4847,
iteration 3:
kmeans: Silhouette Score: 0.1944, ARI Score: 0.3955,CH Score: 58.8866
DBSCAN: Silhouette Score: -0.0647, ARI Score: 0.0000,CH Score: 4.2651
HDBSCAN: Silhouette Score: 0.0993, ARI Score: 0.4836,CH Score: 31.8941
OPTICS: Silhouette Score: -0.1142, ARI Score: 0.0276,CH Score: 8.5230
BisectKMeans: Silhouette Score: 0.2502, ARI Score: 0.4427,CH Score: 95
AgglomerativeClustering: Silhouette Score: 0.2350, ARI Score: 0.4847,
iteration 4:
kmeans: Silhouette Score: 0.1983, ARI Score: 0.3863,CH Score: 56.5718
DBSCAN: Silhouette Score: -0.0647, ARI Score: 0.0000,CH Score: 4.2651
HDBSCAN: Silhouette Score: 0.0993, ARI Score: 0.4836,CH Score: 31.8941
OPTICS: Silhouette Score: -0.1142, ARI Score: 0.0276,CH Score: 8.5230
BisectKMeans: Silhouette Score: 0.2523, ARI Score: 0.4319,CH Score: 96
AgglomerativeClustering: Silhouette Score: 0.2350, ARI Score: 0.4847,
iteration 5:
kmeans: Silhouette Score: 0.2125, ARI Score: 0.4041,CH Score: 59.0749
DBSCAN: Silhouette Score: -0.0647, ARI Score: 0.0000,CH Score: 4.2651
HDBSCAN: Silhouette Score: 0.0993, ARI Score: 0.4836,CH Score: 31.8941
OPTICS: Silhouette Score: -0.1142, ARI Score: 0.0276,CH Score: 8.5230
BisectKMeans: Silhouette Score: 0.2502, ARI Score: 0.4427,CH Score: 95
AgglomerativeClustering: Silhouette Score: 0.2350, ARI Score: 0.4847,
```

Visualizing the results of all clustering algorithms to compare

Box plot for Silhouette score

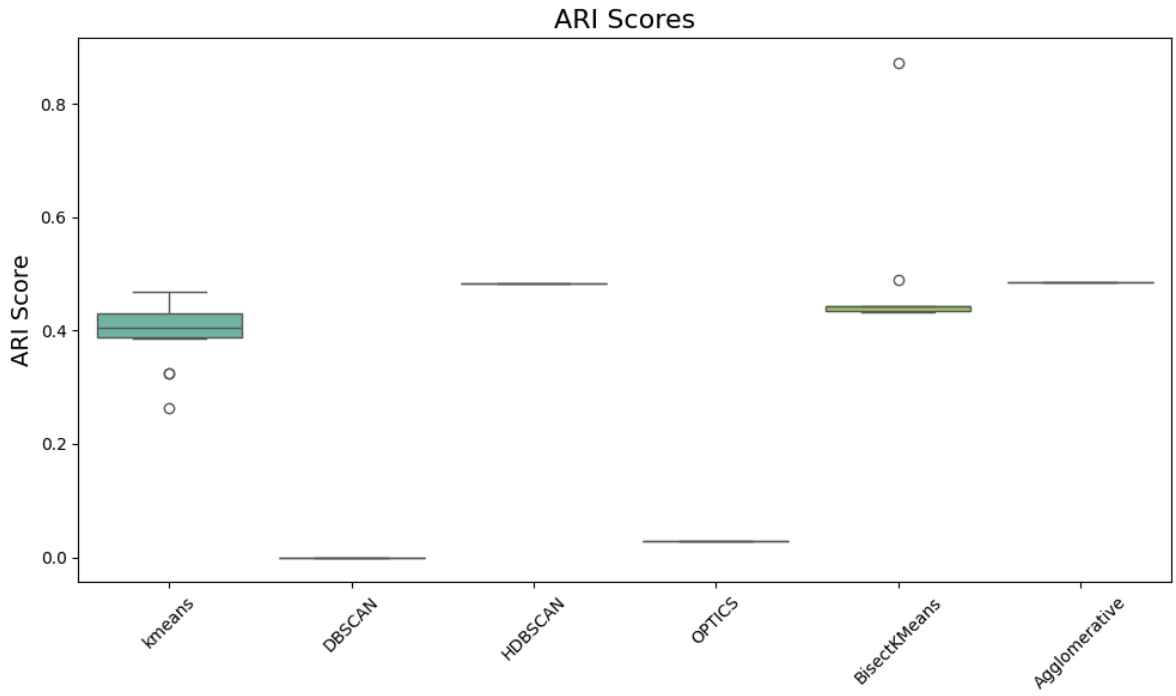
```
plt.figure(figsize=(10, 6))
sns.boxplot(data=silhouette_scores, palette="Set2")
plt.title("Silhouette Scores", fontsize=16)
plt.xticks(rotation=45)
plt.ylabel("Silhouette Score", fontsize=14)
plt.tight_layout()
plt.show()
```



Box plot for ARI score

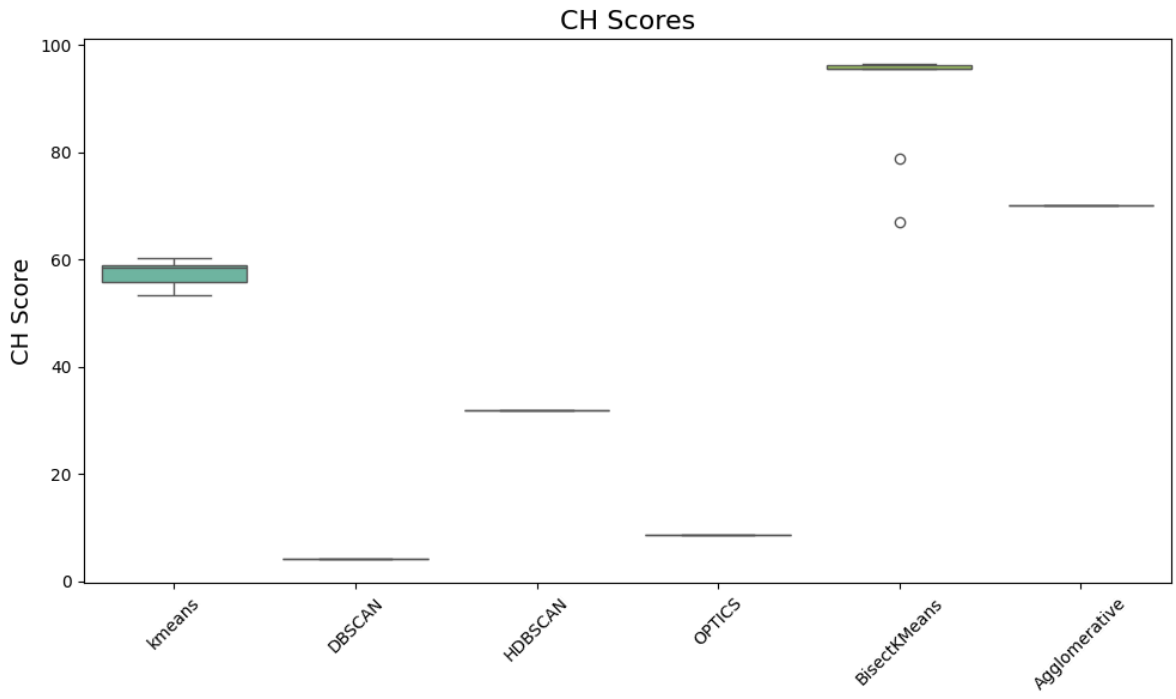
```
plt.figure(figsize=(10, 6))
sns.boxplot(data=ari_scores, palette="Set2")
```

```
plt.title("ARI Scores", fontsize=16)
plt.xticks(rotation=45)
plt.ylabel("ARI Score", fontsize=14)
plt.tight_layout()
plt.show()
```



Box plot for CH score

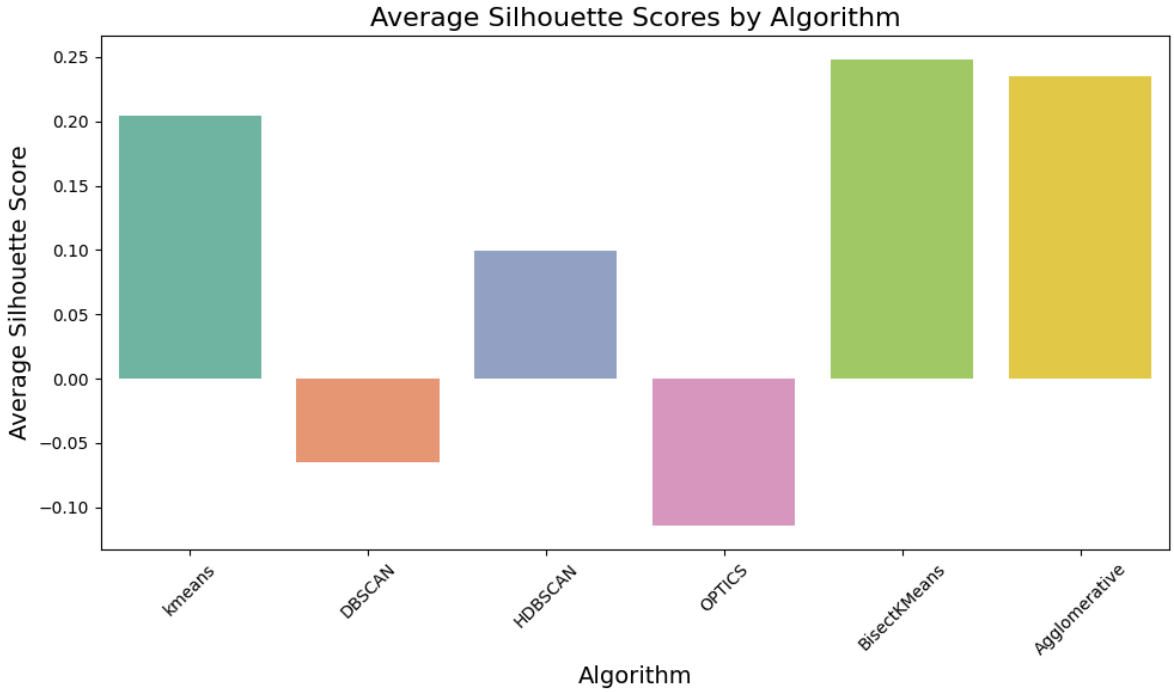
```
plt.figure(figsize=(10, 6))
sns.boxplot(data=CH_scores, palette="Set2")
plt.title("CH Scores", fontsize=16)
plt.xticks(rotation=45)
plt.ylabel("CH Score", fontsize=14)
plt.tight_layout()
plt.show()
```



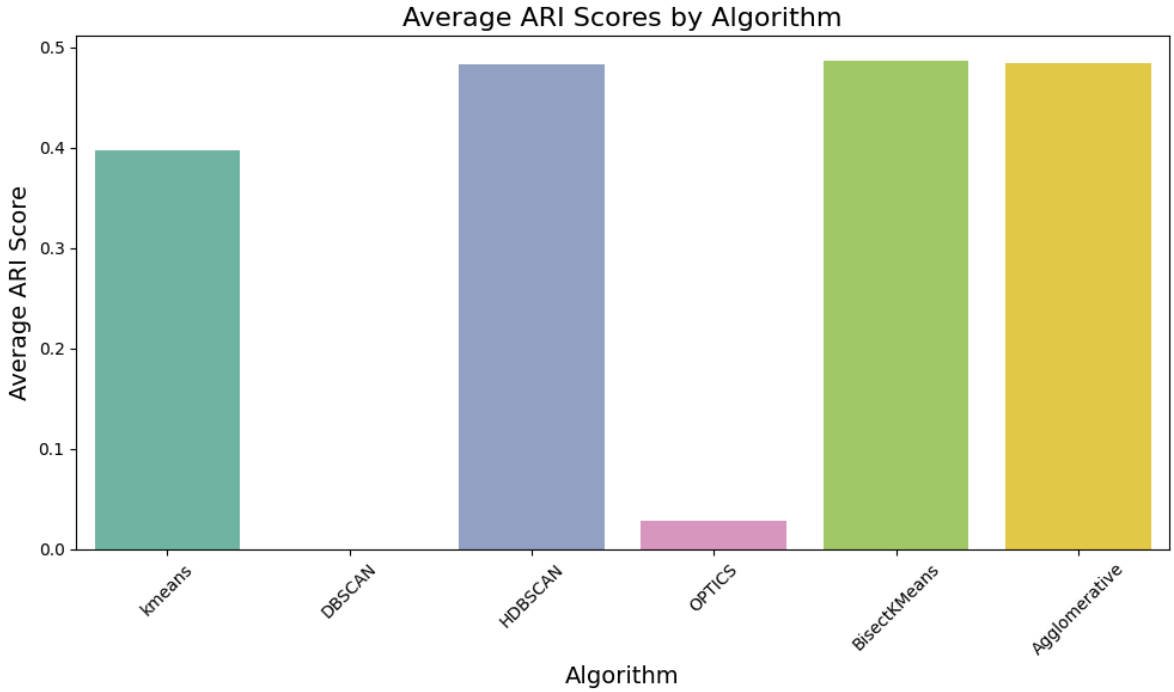
Bar plot for Silhouette score

```
silhouette_avg = {key: np.mean(values) for key, values in silhouette_

plt.figure(figsize=(10, 6))
sns.barplot(x=list(silhouette_avg.keys()), y=list(silhouette_avg.valu
plt.title("Average Silhouette Scores by Algorithm", fontsize=16)
plt.ylabel("Average Silhouette Score", fontsize=14)
plt.xlabel("Algorithm", fontsize=14)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
<ipython-input-147-3a55fdd83ca1>:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be re

  sns.barplot(x=list(silhouette_avg.keys()), y=list(silhouette_avg.val
```



Bar plot for ARI score

```
ari_avg = {key: np.mean(values) for key, values in ari_scores.items()

plt.figure(figsize=(10, 6))
sns.barplot(x=list(ari_avg.keys()), y=list(ari_avg.values()), palette=
plt.title("Average ARI Scores by Algorithm", fontsize=16)
plt.ylabel("Average ARI Score", fontsize=14)
plt.xlabel("Algorithm", fontsize=14)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
<ipython-input-145-ced8cb8ffb45>:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be re

  sns.barplot(x=list(ari_avg.keys()), y=list(ari_avg.values()), palett
```